



GVN.GVNFRAMEWORK 1.0.0-PREVIEW

Framework İnceleme Bulguları

LifeQuest'in framework NuGet paketleri üzerine inşası sırasında tespit edilen hatalar, güvenlik ve tasarım riskleri. Her bulgu için etki, kanıt, LifeQuest'te uygulanan geçici çözüm ve kalıcı düzeltme önerisi yer alır.

İNCELENEN SÜRÜM

1.0.0-preview (commit
b4971d9)

KAPSAM

12 paket · ~3.100 satır

TARİH

24 Eylül 2026

1

Kritik

4

Yüksek

5

Orta

5

Düşük

Bulguların tamamı LifeQuest tarafında framework kaynak koduna dokunulmadan aşılmıştır. Kalıcı çözümler framework'ün bir sonraki `preview` sürümüne aittir.

Kaynak: framework kaynak kodunun okunması, `artifacts/` altındaki NuGet paketlerinin public API dökümü (kaynakla birebir eşleştiği doğrulandı), LifeQuest birim/entegrasyon testleri ve canlı API denemeleri.

Yönetici özeti

Framework, Clean Architecture katmanlaması, `Result<T>` deseni, generic `GvnDbContext<TContext>` ve modüler yapısıyla sağlam bir başlangıç noktası sunuyor. Ancak gerçek bir ürün (LifeQuest) üzerinde kullanıldığında bazı sorunlar ortaya çıktı:

- **Doğrulama:** `Result<T>` dönen her komutta doğrulama hatası 400 yerine **500** üretiyor.
- **Domain event'ler:** event'ler MediatR ile yayınlanamıyor.
- **Loglama:** şifre ve token'lar loglara düşüyor.
- **Hangfire:** kalıcı depolama yapılandırılmıyor ve güvenlik açığı bilinen paketler projeye taşınıyor.

Öneri: 1–5 numaralı bulgular bir sonraki `preview` sürümünde ele alınmalı. Bu sürümde düzeltildiklerinde LifeQuest'teki geçici çözümler (`ResultValidationBehavior` , log maskeleyme politikası, transitive paket sabitlemeleri) kaldırılabilir.

Güçlü yönler

Katmanlama — Core → Domain → Application → Infrastructure ayrımı net; paketler bağımsız kullanılabilir.

Result deseni — Tipli `Error` / `ErrorType` ile exception'sız iş akışı.

Generic DbContext — `GvnDbContext<TContext>` audit, soft-delete filtresi ve event dağıtımını tek yerde topluyor.

Genişletilebilirlik — `IModule` , `Decorate` , `Specification` ve cache soyutlaması yeniden kullanılabilir.

Bulgu listesi

#	ÖNEM	PAKET	BULGU
1	KRİTİK	Application	<code>ValidationBehavior</code> hatalı cast: doğrulama hataları 500 oluyor
2	YÜKSEK	Domain · EF	<code>IDomainEvent</code> , <code>INotification</code> değil; event yayını çalışmıyor
3	YÜKSEK	EntityFrameworkCore	Event'ler commit sonrası dağıtılıyor; Outbox tanımlı ama kullanılmıyor
4	YÜKSEK	Application	Request/response'lar olduğu gibi loglanıyor (PII, şifre, token)
5	YÜKSEK	BackgroundJobs	Kalıcı storage yok; güvenlik açığı bilinen transitive paketler
6	ORTA	AspNetCore · Logging	Correlation id loglara taşınmıyor, doğrulanmıyor
7	ORTA	EntityFrameworkCore	<code>OutboxConfiguration</code> SQL Server'a özgü tip kullanıyor
8	ORTA	Security	Soyutlama/uygulama ayrımı yok; refresh token ve ayar doğrulaması yok
9	ORTA	Swagger	<code>UseGvnSwagger</code> middleware sırasını gizlice değiştiriyor
10	ORTA	AspNetCore	Hata yanıtı biçimleri tutarsız; <code>Failure</code> → 500, 403/429 yok
11	DÜŞÜK	Tümü	Paket adlarında yazım hatası (<code>EntityFrameworkCore</code> , <code>DepedencyInjection</code>)
12	DÜŞÜK	Tümü	<code>net10.0</code> hedefinde 9.x Microsoft paketleri

#	ÖNEM	PAKET	BULGU
13	DÜŞÜK	EF · Domain	EfRepository context'i açmıyor; audit alanlarında kullanıcı yok
14	DÜŞÜK	Modularity	Statik modül listesi; IModule konfigürasyon almıyor
15	DÜŞÜK	Paketleme	Aynı sürümle yeniden paketleme NuGet önbeliğinde kayboluyor

Bulgu detayları

01 ValidationBehavior hatalı cast yapıyor

Application

KRİTİK

Sorun	Doğrulama hatasında Result.Fail(...) ile oluşturulan temel Result nesnesi TResponse'a cast ediliyor. TResponse, Result<T> olduğunda (ICommand<T> / IQuery<T>) bu cast geçersiz.
Etki	Veri döndüren her komutta istemci hatası (400) yerine InvalidCastException → 500. Doğrulama pratikte yalnızca ICommand (veri döndürmeyen) için çalışıyor. Ayrıca kurallar senkron Validate ile çalıştığı için async kurallar hata fırlatıyor.
Kanıt	<pre>var result = Result.Fail(validationErrors); // temel Result örneği return (TResponse)result; // TResponse = Result<Guid> → InvalidCastException</pre>
Geçici çözüm	LifeQuest'te ResultValidationBehavior framework behavior'ının pipeline'daki yerine konuldu: Result<T>.Fail doğru tipte oluşturuluyor, ValidateAsync kullanılıyor. Entegrasyon testiyle doğrulandı.
Kalıcı çözüm	Başarısız sonucu tipine göre üreten bir fabrika kullanılmalı (örn. static abstract arayüz üyesi IResultFactory<TSelf>.Failure(errors) ya da önbeliklenmiş reflection). Doğrulama ValidateAsync ile yapılmalı. Her iki dönüş tipi için birim testi eklenmeli.

02 IDomainEvent, INotification değil

Domain · EF

YÜKSEK

Sorun	GvnDbContext event'leri IMediator.Publish(object) ile yayınlıyor. MediatR bu overload'da nesnenin INotification olmasını şart koşuyor; framework'ün IDomainEvent / DomainEvent tipleri bunu uygulamıyor.
Etki	Framework tabanından türetilen her domain event, SaveChangesAsync sırasında (veri commit edildikten sonra) exception fırlatır.
Geçici çözüm	LifeQuestDomainEvent : DomainEvent, INotification ortak tabanı.
Kalıcı çözüm	IDomainEvent : INotification yapılmalı (Domain paketi yalnızca MediatR.Contracts'a bağımlı olur) ya da domain'i MediatR'dan bağımsız tutmak için DomainEventNotification<TEvent> sarmalayıcısıyla yayın yapılmalı.

03 Domain event'ler commit sonrası, transaction dışında dağıtılıyor

EntityFrameworkCore

YÜKSEK

Sorun	Event'ler base.SaveChangesAsync tamamlandıktan sonra aynı çağrının içinde yayınlıyor. OutboxMessage ve OutboxConfiguration tanımlı ama hiçbir yerde
-------	---

kullanılmıyor.

Etki

Handler'da oluşan hata, veri başarıyla yazılmış olsa bile çağırana exception olarak döner; istemci 500 alır ve isteği tekrarlayabilir (çift işlem riski). Handler'ın yan etkisi (e-posta, bildirim) kaybolabilir; yeniden deneme mekanizması yok.

Geçici çözüm

LifeQuest'te kritik yan etkiler (XP, başarımlar, ilgi öğrenmesi) aynı transaction'da handler içinde yapılıyor; event handler'ları yalnızca log yazıyor.

Kalıcı çözüm

Outbox deseni tamamlanmalı: `SaveChangesAsync` event'leri aynı transaction'da `OutboxMessages` tablosuna yazmalı, arka plan işleyicisi (Hangfire/hosted service) bunları yeniden deneme ve idempotency ile yayınlamalı.

04 Request ve response'lar olduğu gibi loglanıyor

Application

YÜKSEK

Sorun

`LoggingBehavior` her isteği ve yanıtı, `PerformanceBehavior` ise yavaş istekleri `{@Request}` / `{@Response}` ile tüm alanlarıyla yapılandırılmış log olarak yazıyor.

Etki

Giriş/kayıt komutlarındaki **şifreler**, yanıtlardaki **access/refresh token'lar** ve e-posta gibi kişisel veriler log dosyalarına ve log toplama sistemlerine düşer. KVKK/GDPR ve güvenlik açısından ciddi risk.

Kanıt

```
logger.LogInformation("Handling {RequestName}: {@Request}", requestName, request);
logger.LogInformation("Handled {RequestName}: {@Response}", requestName, response);
```

Geçici çözüm

Serilog `IDestructuringPolicy` ile `*Password*`, `*Token*`, `*Secret*` alanları tamamen, `Email` kısmen maskeleniyor (canlı loglarda doğrulandı).

Kalıcı çözüm

Varsayılan olarak yalnızca istek adı, süre ve sonuç (başarılı/hata kodu) loglanmalı. Gövde loglaması isteğe bağlı olmalı (`[LogPayload]` attribute'u veya ayar), hassas alanlar için `[Sensitive]` attribute'u ve yerleşik maskeleme sunulmalı.

05 BackgroundJobs: kalıcı storage yok, güvenlik açıklı bağımlılıklar

BackgroundJobs

YÜKSEK

Sorun

- `UseInMemory = false` olduğunda hiçbir storage yapılandırılmıyor; `SqlServerConnectionString` seçeneği okunmuyor. PostgreSQL desteği yok.
- `Hangfire.RecurringJobAdmin 1.0.5`, eski `Hangfire 1.7.18` metapaketini ve onunla `Newtonsoft.Json 11.0.1` (GHSA-5crp-9r3c-p9vr, yüksek) ile `System.Data.SqlClient 4.4.0` (GHSA-8g2p-5pqh-5jmc, GHSA-98g6-xh36-x2p7) paketlerini getiriyor.

Etki

Üretimde InMemory dışında çalıştırılmıyor; uygulama yeniden başlayınca job durumu kaybolur. Tüketen her projeye NU1903/NU1608 uyarıları ve bilinen açıklar taşıyor.

Geçici çözüm

InMemory storage + idempotent job'lar. `CentralPackageTransitivePinningEnabled` ile Hangfire 1.8.17, Newtonsoft.Json 13.0.3, System.Data.SqlClient 4.9.0 sabitlendi.

Kalıcı çözüm

Storage için genişletme noktası (`Action<IGlobalConfiguration> configureStorage`) eklenmeli, `SqlServerConnectionString` gerçekten uygulanmalı veya kaldırılmalı, `Hangfire.PostgreSql` desteği eklenmeli. `RecurringJobAdmin` çıkarılmalı ya da güncel

sürüme sabitlenmeli; pakete `dotnet list package --vulnerable` kontrolü CI'da eklenmeli.

06 Correlation id loglara taşınmıyor ve doğrulanmıyor

AspNetCore · Logging

ORTA

Sorun `CorrelationIdMiddleware` kimliği yalnızca `HttpContext.Items` 'a ve yanıt header'ına yazıyor; Serilog `LogContext` 'e eklemiyor. İstemciden gelen değer uzunluk/karakter kontrolü olmadan kabul ediliyor. `CorrelationIdEnricher` sabit id verilmezse her log satırına **farklı** rastgele id üretiyor.

Etki Log şablonundaki `{CorrelationId}` boş kalıyor; bir isteğin logları ilişkilendirilemiyor. Header ile log enjeksiyonu yapılabilir.

Geçici çözüm `CorrelationIdLogContextMiddleware` : değeri temizliyor (en fazla 64 karakter, `[A-Za-z0-9_-]`), `LogContext` 'e ve `Activity` etiketine ekliyor.

Kalıcı çözüm Middleware içinde doğrulama + `LogContext.PushProperty` ; enricher `IHttpContextAccessor` 'dan okumalı ya da kaldırılmalı. W3C `traceparent` ile hizalanması değerlendirilmeli.

07 OutboxConfiguration SQL Server'a özgü

EntityFrameworkCore

ORTA

Sorun `Content` kolonu `HasColumnType("nvarchar(max)")` ile tanımlı.

Etki PostgreSQL, SQLite gibi sağlayıcılarda migration başarısız olur; framework'ün "sağlayıcıdan bağımsız EF katmanı" iddiasıyla çelişir.

Kalıcı çözüm Kolon tipi belirtilmemeli (sağlayıcı varsayılanı) veya sağlayıcıya göre koşullu yapılandırılmalı; PostgreSQL için `jsonb` seçeneği sunulabilir.

08 Security paketi: katman ayrımı, refresh token ve ayar doğrulaması eksik

Security

ORTA

Sorun

- `IPasswordHasher` , `ITokenService` gibi soyutlamalar ASP.NET Core bağımlı implementasyonlarla aynı pakette; Application katmanı web framework'üne bağımlı hale geliyor.
- Refresh token desteği yok; kısa ömürlü access token pratikte kullanılmıyor.
- `JwtOptions` doğrulanmıyor; boş ya da kısa `Secret` ancak ilk token üretiminde hata veriyor.

Geçici çözüm LifeQuest'te rotasyonlu, hash'lenmiş refresh token (çalınma tespitiyle), hesap kilitleme ve açılışta en az 32 bayt secret kontrolü.

Kalıcı çözüm `Gvn.GvnFramework.Security.Abstractions` paketi ayrılmalı; `AddOptions<JwtOptions>().ValidateDataAnnotations().ValidateOnStart()` ; refresh token soyutlaması (store + rotation) framework'e alınmalı.

09 UseGvnSwagger middleware sırasını gizlice değiştiriyor

Swagger

ORTA

Sorun Metot içeride `UseRouting()` ve `UseEndpoints()` çağırıyor. Örnek projede

	<code>UseAuthentication/UseAuthorization</code> bu çağrıdan önce olduğundan, yetkilendirme routing'den önce çalışıyor.
Etki	<code>[Authorize]</code> uçları eklendiğinde "authorization middleware bulunamadı" hatası veya beklenmeyen davranış. Çağrı sırası dışarıdan görülemiyor.
Kalıcı çözüm	<code>IEndpointRouteBuilder.MapGvnSwagger()</code> uzantısı sunulmalı (<code>app.MapOpenApi()</code> + <code>MapScalarApiReference()</code>), routing çağrıları kaldırılmalı; örnek proje sırası düzeltilmeli.

10 Hata yanıtı biçimleri tutarsız

AspNetCore

ORTA

Sorun	<code>ApiControllerBase</code> hataları <code>Error[]</code> olarak, <code>ExceptionHandlerMiddleware</code> ise <code>problem+json</code> olarak döndürüyor; model binding hataları üçüncü bir biçimde (<code>ValidationProblemDetails</code>). <code>ErrorType.Failure</code> 500'e çevriliyor; <code>Forbidden</code> / <code>TooManyRequests</code> tipi yok.
Etki	İstemciler üç farklı hata gövdesini ayrıştırmak zorunda (LifeQuest web istemcisinde <code>parseApiErrors</code> bunun için yazıldı). İş kuralı hatası yanlışlıkla <code>Failure</code> ile dönülürse sonucu hatası gibi görünür.
Kalıcı çözüm	Tek biçim: RFC 9457 <code>ProblemDetails</code> + <code>errors</code> uzantısı (<code>code</code> , <code>message</code>). <code>ErrorType</code> 'a <code>Forbidden</code> eklenmeli; <code>Failure</code> 400/422'ye eşlenmeli.

11–15 Düşük öncelikli bulgular

DÜŞÜK

- Paket adları:** `EntityFrameworkCore`, `DependencyInjection`. Namespace'ler de aynı yazımla. NuGet'e yayınlanmadan önce düzeltilmeli; sonrası tüm tüketiciler için kırıcı değişiklik olur.
- Sürüm hizalaması:** `net10.0` hedeflenirken EF Core 9.0.4, `Microsoft.AspNetCore.OpenApi` 9.0.4, `JwtBearer` 9.0.4 ve `Caching.StackExchangeRedis` 9.0.4 kullanılıyor; 10.x ile hizalanmalı. (LifeQuest, Npgsql 9.0.4 ile uyumlu tutuldu.)
- Repository ve audit:** `EfRepository` context'i `protected` olarak açmıyor (türeyen repository'ler diğer DbSet'lere erişemiyor); `GetByIdAsync` child koleksiyonları yüklemiyor. `GvnDbContext` audit için `SetCreated(null)` çağırıyor; `CreatedBy` / `UpdatedBy` hiç dolmuyor. Öneri: `protected TContext Context` ve `IAuditUserProvider` kancası.
- Modularity:** `ModuleLoader` statik liste kullanıyor (testlerde birden fazla host oluşturulunca modüller birikiyor); `IModule.ConfigureServices` `IConfiguration` almıyor ve modüllerin parametresiz constructor'ı olmak zorunda. Öneri: örnek bazlı kayıt ve `ConfigureServices(IServiceCollection, IConfiguration)`.
- Paketleme:** Değişiklikler aynı `1.0.0-preview` sürümüyle yeniden paketlenildiğinde NuGet genel önbellesi eski paketi kullanmaya devam ediyor (`artifacts` içindeki EF paketi diğerlerinden farklı tarihte üretilmiş). Öneri: her derlemede artan sürüm (`preview.N` / git yüksekliği) ve CI'dan paket yayını.

Ek gözlemler:

- `PerformanceBehavior` eşiği (500 ms) constructor parametresi olarak sabit; yapılandırılamıyor.
- `AddGvnLogging(IHostBuilder)` logger'ı iki kez oluşturuyor.
- `MemoryCacheService`, Redis uygulamasının aksine anahtar önekini (`Prefix`) kullanmıyor.

Önerilen eylem planı

SÜRÜM	KAPSAM	KAZANIM
1.0.1-preview ACİL	#1 ValidationBehavior düzeltmesi · #2 IDomainEvent : INotification · #4 varsayılan log davranışının güvenli hale getirilmesi · #5 RecurringJobAdmin'in çıkarılması/sabitlenmesi	500 hataları ve PII sızıntısı biter; bilinen açıklar kalkar
1.1.0-preview	#3 Outbox'ın tamamlanması · #5 storage genişletme noktası + PostgreSQL · #6 correlation id · #7 sağlayıcıdan bağımsız outbox · #10 ProblemDetails birleştirmesi	Güvenilir event'ler, üretimde Hangfire, izlenebilirlik
1.2.0-preview	#8 Security.Abstractions + refresh token · #9 MapGvnSwagger · #12 10.x hizalaması · #13 audit kullanıcı kancası · #14 Modularity iyileştirmeleri	Temiz katman bağımlılıkları, daha az şaşırtıcı API
2.0.0 KIRICI	#11 paket/namespace adlarının düzeltilmesi · #15 otomatik sürümlleme ve CI yayını · birim/entegrasyon test projesinin eklenmesi	NuGet'e yayına hazır, kalıcı adlar

LifeQuest'te kaldırılabilir geçici çözümler

GEÇİCİ ÇÖZÜM	KONUM	KALDIRMA KOŞULU
ResultValidationBehavior	LifeQuest.Application/Behaviors	#1 düzeltildiğinde
LifeQuestDomainEvent ortak tabanı	LifeQuest.Domain/Common	#2 düzeltildiğinde
SensitiveDataDeconstructingPolicy	LifeQuest.Api/Infrastructure	#4 düzeltildiğinde
Transitive paket sabitlemeleri	Directory.Packages.props	#5 düzeltildiğinde
CorrelationIdLogContextMiddleware	LifeQuest.Api/Infrastructure	#6 düzeltildiğinde
Repository'lerde ayrı context alanı	LifeQuest.Infrastructure/Persistence/Repositories	#13 düzeltildiğinde

Doğrulama yöntemi

- Framework kaynak kodu satır satır okundu; artifacts/*.nupkg içindeki DLL'lerin public API'si çıkarılıp kaynakla karşılaştırıldı (NuGet ön belleğiyle birebir aynı).
- #1, #2, #4, #6 bulguları LifeQuest'te gözlemlendi; geçici çözümler **46 domain + 11 entegrasyon testi** (gerçek PostgreSQL) ve canlı API denemeleriyle doğrulandı.
- #5'teki açıklar dotnet build NuGet denetim uyarılarından (NU1902/NU1903) alındı.

Ayrıntılı ürün analizi değerlendirmesi ve LifeQuest'e eklenen katkılar: docs/analiz-degerlendirmesi.md
— github.com/gvnuysal/gvn.gvnai.lifequest